

GITOPS-BASED KUBERNETES DEPLOYMENT

Internship Project Report

1. Introduction

This project presents an end-to-end DevOps and GitOps workflow for deploying a Python Flask web application. The main objective was to build a small but complete deployment pipeline in which application changes are validated automatically, packaged as Docker images, checked for security vulnerabilities, published to a registry, and delivered to Kubernetes through Argo CD. GitHub is used as the central source-control and desired-state repository, while Minikube provides the local Kubernetes environment used for deployment and testing.

The project also focuses on demonstrating how GitOps improves consistency and recovery. Instead of treating the running Kubernetes cluster as the primary source of deployment configuration, the Kubernetes manifests stored in Git define the intended state. Argo CD continuously compares that state with the cluster and synchronizes differences. This approach was tested through application updates, an invalid deployment image, rollback, manual pod deletion and automatic recovery.

2. Abstract

A lightweight Flask application was created with a versioned response and exposed on port 5000. The application is containerized using Docker and deployed through a Kubernetes Deployment with two replicas and a NodePort Service. When code is pushed to the main branch, GitHub Actions executes automated tests, builds the Docker image using the Git commit SHA as the image tag, scans the image with Trivy, pushes the validated image to Docker Hub, and updates the Kubernetes Deployment manifest with the new image reference.

Argo CD monitors the k8s directory of the GitHub repository and automatically synchronizes changes into the gitops-demo namespace. Automated prune and self-heal are enabled. The completed implementation demonstrated the complete path from source-code change to a running Kubernetes application, as well as recovery and rollback using Git-based changes.

3. Tools Used

Tool / Technology	Purpose in the Project
Python / Flask	Developed the versioned web application.
Git / GitHub	Stored source code, manifests and complete project history.
GitHub Actions	Automated tests, Docker build, security scan and manifest update.
Docker	Containerized the Flask application.
Docker Hub	Stored the application container images.
Trivy	Scanned the container image for HIGH and CRITICAL OS vulnerabilities.
Kubernetes	Ran the application using Deployment, ReplicaSets, Pods and Service.
Argo CD	Provided GitOps synchronization and self-healing.
Minikube	Provided the local Kubernetes cluster.

4. Steps Involved in Building the Project

The project was built as a sequence of connected stages so that each tool had a clear responsibility in the final workflow. First, the Flask application was created with a simple root endpoint. The application version was kept in the source code so that a change from v1 to v2 could be visibly verified after deployment. A Dockerfile was then used to package the application and its Python dependencies into a reproducible container image.

The Kubernetes configuration was created under the k8s directory. The Deployment specifies two replicas and exposes container port 5000. The Service uses NodePort and selects the application pods using the app=gitops-demo label. An Argo CD Application was configured to monitor the GitHub repository, use the k8s path, target the main branch and deploy into the gitops-demo namespace.

The CI/CD workflow was implemented in .github/workflows/ci.yml. The test job installs the application requirements using Python 3.14 and executes a Flask endpoint test. The Docker job runs only after the test job succeeds. It builds the image with the Git commit SHA, verifies important Python package versions, runs Trivy, logs in to Docker Hub and pushes the image. Finally, it updates deployment.yaml with the newly built image tag and commits that manifest change back to GitHub.

This creates the GitOps chain: a developer changes application code, GitHub Actions validates and packages it, the new image is published, the Kubernetes desired state is updated in Git, and Argo CD detects that Git change and reconciles the cluster. Therefore, the deployment configuration remains version controlled and traceable to Git commits.

5. Testing, Failure Scenarios and Results

Testing was performed to verify not only successful deployment but also application updates, deployment failure, rollback and Kubernetes recovery. The initial deployment was verified with two running application pods, a working NodePort Service and an Argo CD application reported as Synced and Healthy. This established the baseline state for the remaining tests.

Application version update: The Flask application was changed from version v1 to v2. The change was committed and pushed to the main branch. GitHub Actions completed successfully, the Docker image was created and published, and the Kubernetes manifest was updated automatically. After Argo CD synchronization, the application returned **GitOps Demo Application - Version v2**. This verified the complete code-to-deployment flow.

Failed deployment scenario: The Kubernetes manifest was deliberately changed to reference the invalid image tag **shohithdocker/gitops-demo:invalid-failure-test**. The change was committed to Git so that Argo CD would process it as a real desired-state change. Kubernetes showed the resulting deployment behavior through pod and deployment status. The test demonstrated how an incorrect image configuration can be introduced and then recovered through a controlled Git change.

Rollback: After the v2 test, the application was restored to v1 and the corresponding valid image was placed back into the Kubernetes manifest. The Git change was pushed and synchronized by Argo CD. The final application response confirmed **GitOps Demo Application - Version v1**. The Argo CD history also showed multiple deployment revisions, providing a record of the states through which the application had progressed.

Pod failure and recovery: One running application pod was manually deleted using `kubectl`. Because the Deployment specifies two replicas, Kubernetes automatically created a replacement pod. The replacement became Ready, and the final state again contained two running replicas. Service endpoints also remained available, demonstrating the recovery behavior provided by the Kubernetes Deployment controller.

Argo CD recovery and synchronization: The project repeatedly verified the Argo CD Application using `kubectl get application gitops-demo -n argocd`. The final state reported **Synced** and **Healthy**. The deployed image was also checked directly from the Deployment specification, and the application response was tested from inside the running deployment. These checks confirmed that Git, Argo CD and Kubernetes had converged on the intended state.

6. Security and Reliability Practices

Security was included as part of the CI/CD process rather than being treated as a separate manual activity. Trivy was integrated into the Docker job and configured to scan for HIGH and CRITICAL operating-system vulnerabilities. The workflow uses an exit code that can fail the pipeline when applicable findings are detected, while ignore-unfixed is enabled so that unresolved upstream issues do not unnecessarily block the demonstration pipeline. Python package versions inside the image were also explicitly verified during the security configuration work.

Reliability was demonstrated through Kubernetes replica management and Argo CD reconciliation. Maintaining two replicas reduces the impact of losing a single pod, while the Deployment controller automatically replaces a deleted pod. Argo CD provides another recovery layer by continuously comparing the cluster with the Git-defined desired state. Together, these mechanisms demonstrate both workload recovery and configuration reconciliation.

7. Conclusion

The project successfully demonstrates a practical GitOps-based Kubernetes deployment pipeline suitable for an internship-level DevOps project. It connects source control, automated testing, Docker containerization, security scanning, image publishing, Kubernetes orchestration and Argo CD continuous delivery into one workflow. The use of Git commit SHA image tags also makes deployed versions traceable to source changes.

The testing phase provided evidence that the system can handle normal releases as well as controlled failure and recovery scenarios. The v1 to v2 update confirmed automated delivery, the invalid-image test demonstrated failure handling, the rollback restored a known working version, and pod deletion demonstrated Kubernetes self-recovery. Argo CD ultimately reported the application as Synced and Healthy, confirming that the cluster matched the desired state stored in Git.

8. Core Project Boundary

The project is intentionally limited to demonstrating CI/CD, Docker, container security scanning, Docker Hub, Kubernetes, Minikube, Argo CD GitOps synchronization, self-healing, failure testing and rollback. It does not cover production-scale cloud provisioning, multi-cluster management, enterprise secret management, advanced observability or high-availability infrastructure.